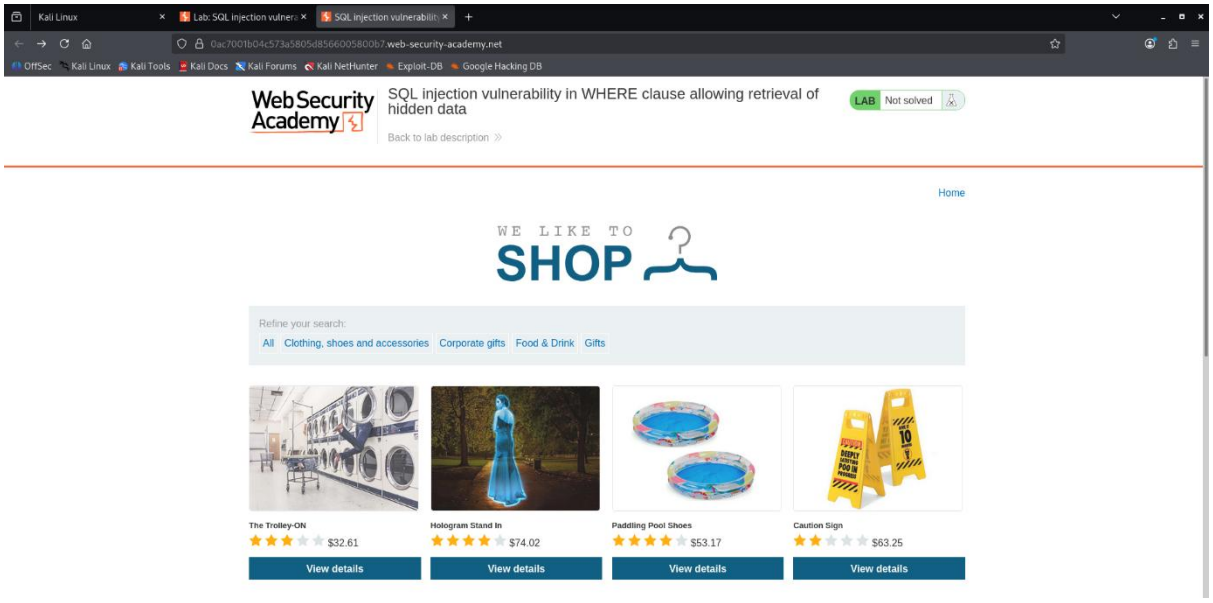
DAY 28 PRACTICAL LAB — SQL INJECTION

Platform: PortSwigger Web Security Academy

Lab: SQL injection vulnerability in WHERE clause allowing retrieval of hidden data

STEP 1
Open the PortSwigger lab from the Resources section.

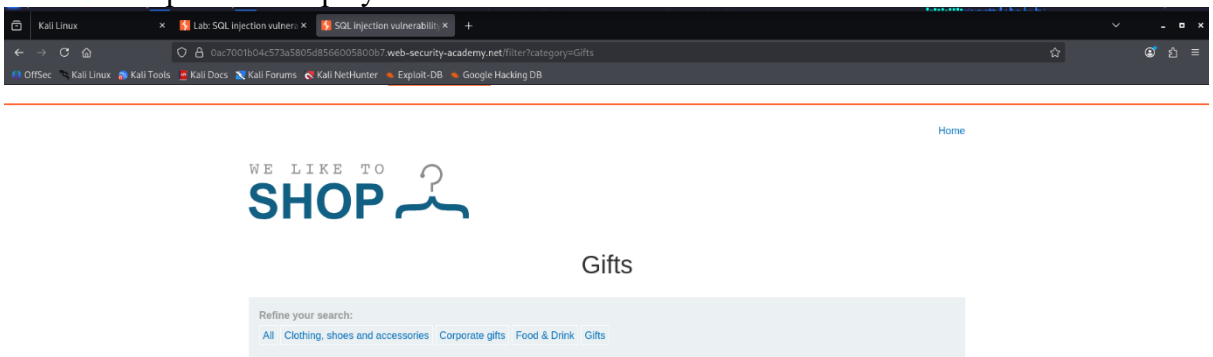
Click: ACCESS THE LAB



STEP 2
Inside the vulnerable shopping application, select a product category such as:

Gifts

Observe the products displayed.



STEP 3

Open Burp Suite Community Edition.

Use Burp's built-in browser if required.

STEP 4

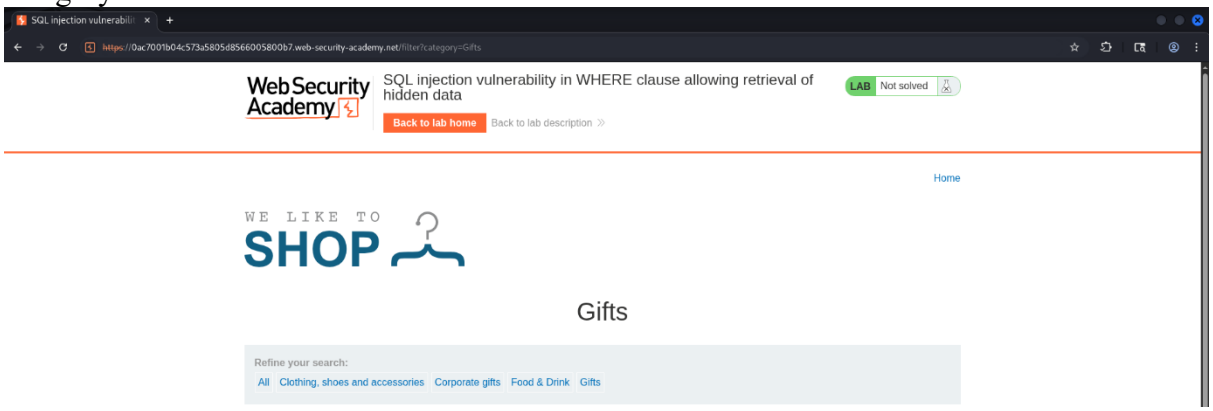
Open:

Proxy → HTTP History

Find the request containing the category parameter.

Example:

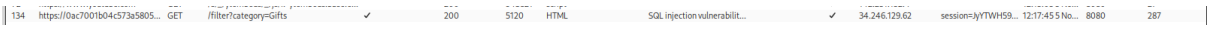
category=Gifts



STEP 5

Select the request and choose:

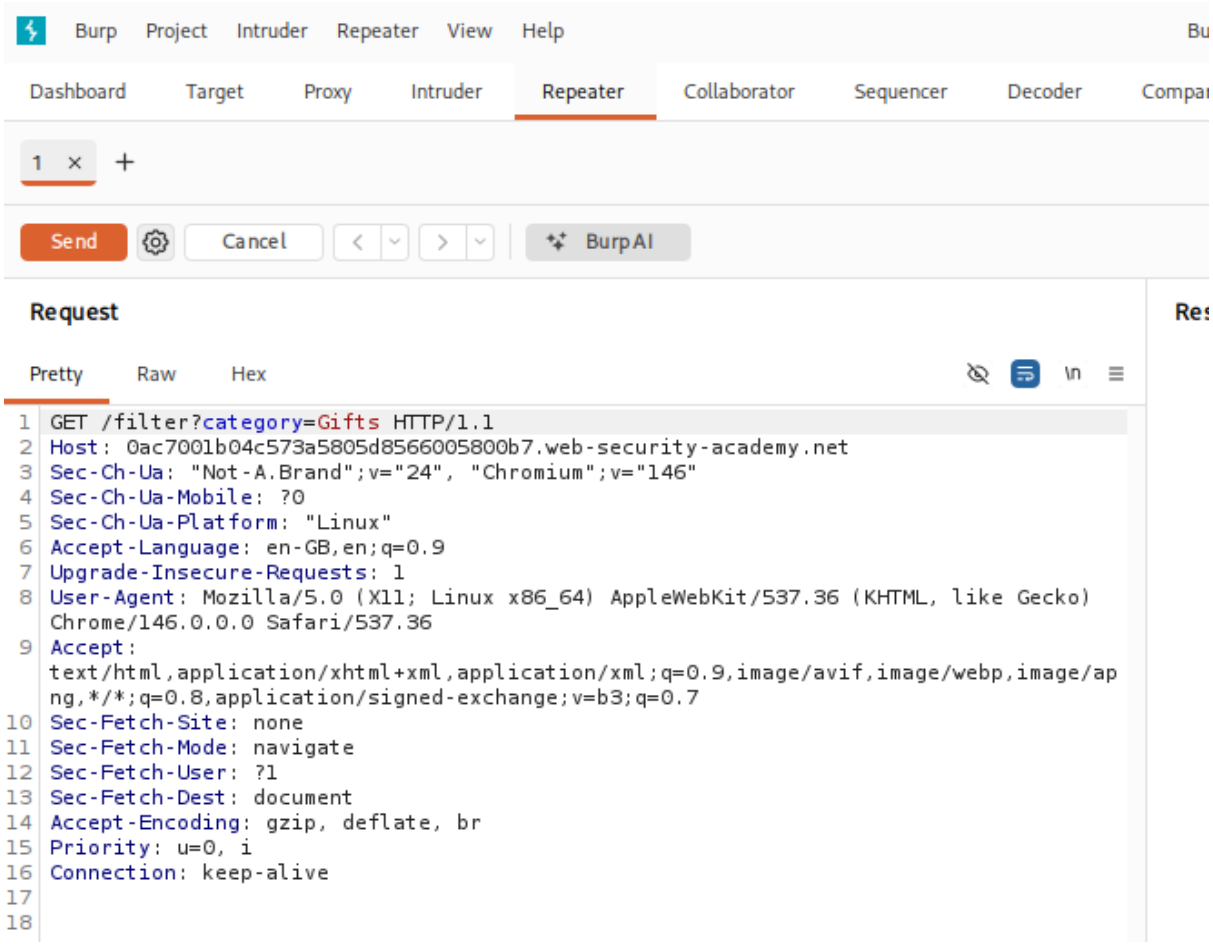
Send to Repeater



STEP 6

Open Repeater.

Locate the category parameter.



STEP 7

ONLY inside this authorized PortSwigger lab, change the category value using the training input:

' OR 1=1—

STEP 8

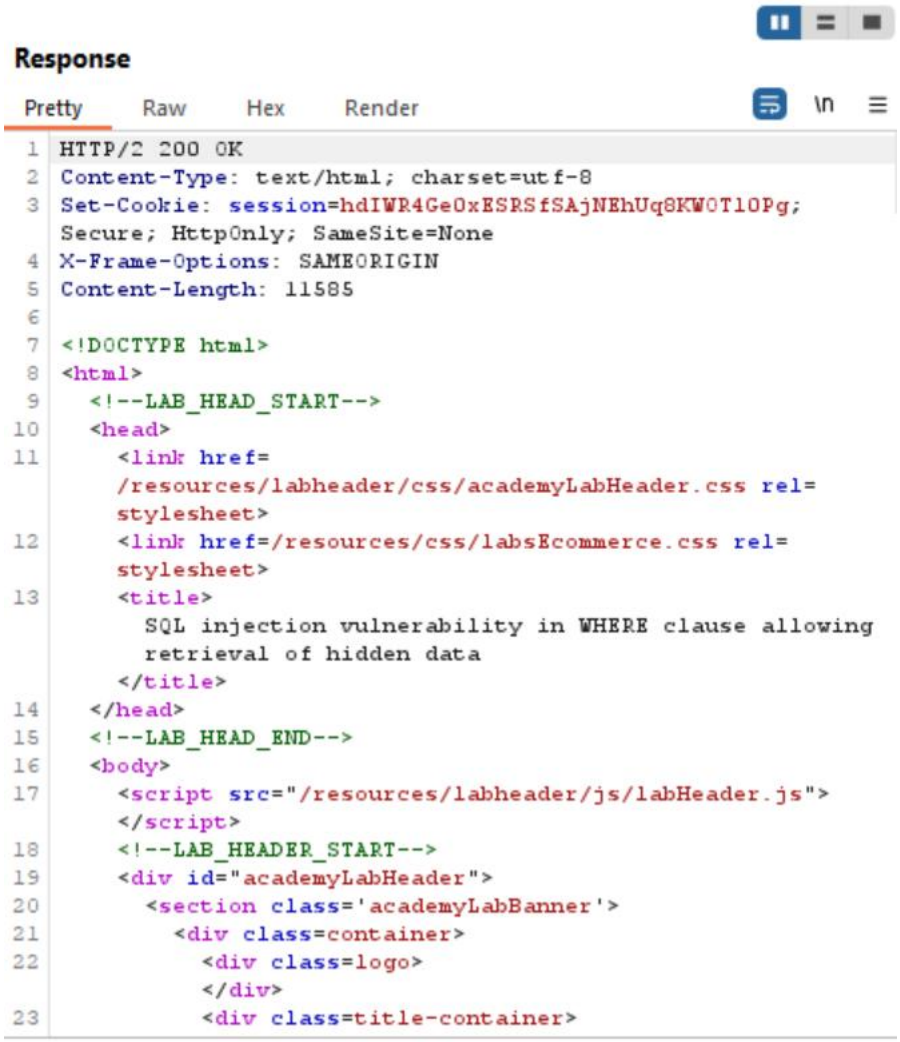
Send the request.

Compare the response with the original request.



STEP 9

Observe whether additional products, including unreleased products, become visible.



Response

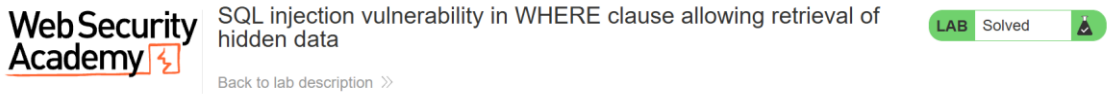
Pretty Raw Hex Render \n

```
1  HTTP/2 200 OK
2  Content-Type: text/html; charset=utf-8
3  Set-Cookie: session=hdIWR4Ge0xESRSfSAjNEhUq8KW0T1OPg;
   Secure; HttpOnly; SameSite=None
4  X-Frame-Options: SAMEORIGIN
5  Content-Length: 11585
6
7  <!DOCTYPE html>
8  <html>
9    <!--LAB_HEAD_START-->
10   <head>
11     <link href=
       /resources/labheader/css/academyLabHeader.css rel=
       stylesheet>
12     <link href=/resources/css/labsEcommerce.css rel=
       stylesheet>
13     <title>
         SQL injection vulnerability in WHERE clause allowing
         retrieval of hidden data
       </title>
14   </head>
15   <!--LAB_HEAD_END-->
16   <body>
17     <script src="/resources/labheader/js/labHeader.js">
       </script>
18     <!--LAB_HEADER_START-->
19     <div id="academyLabHeader">
20       <section class='academyLabBanner'>
21         <div class=container>
22           <div class=logo>
             </div>
23           <div class=title-container>
```

STEP 10

Complete the lab until PortSwigger displays:

Solved



STEP 11

Record:

- Vulnerable parameter: Category
- HTTP method: GET
- Training input used: ' OR 1=1--
- What changed in the response: Server returned additional products, including unreleased products.
- Why the query behaviour changed: OR 1=1 introduced always a true condition, while – commenting out the remaining SQL condition.

- Main defensive control: Parameterized Queries